

ANDRYUS LAB

Especificación Técnica v1.0 — Documento de implementación

Objetivo: convertir el Documento Maestro en una guía suficientemente precisa para que un desarrollador o una IA pueda crear el primer MVP sin tener que reinterpretar la arquitectura fundamental.

▢ Versión: 1.0 • Fecha: 21 de septiembre de 2026

△ Esta especificación define decisiones iniciales. Las cuotas y condiciones de proveedores deben verificarse nuevamente al momento del despliegue.

1. Alcance del MVP

El MVP no intentará construir toda la plataforma. Debe demostrar la arquitectura con pocas herramientas y dejar preparada la infraestructura para crecer.

- Frontend estático, responsive y mobile-first.
- Sistema modular de herramientas.
- Tres herramientas iniciales sin backend.
- Una herramienta temporal con persistencia.
- Autenticación opcional, no obligatoria para usar herramientas públicas.
- Adaptadores independientes para Firebase y Supabase.
- Pruebas unitarias de la lógica de dominio.
- Despliegue automatizado desde GitHub.

2. Stack recomendado

Capa	Tecnología propuesta	Razón
Frontend	Vite + TypeScript	Ligero, rápido y modular.
UI	HTML/CSS + componentes propios	Evita dependencia innecesaria.
Estado local	LocalStorage / IndexedDB	Persistencia sin servidor.
Backend A	Firebase	Auth y funciones puntuales.
Backend B	Supabase	PostgreSQL/API cuando exista necesidad relacional.
Tests	Vitest	Pruebas rápidas de lógica.
Lint/format	ESLint + Prettier	Consistencia del código.
CI/CD	GitHub Actions	Build y tests automáticos.

3. Estructura exacta del repositorio

```
andryus-lab/  
├── apps/web/  
│   ├── src/  
│   │   ├── app/  
│   │   ├── components/  
│   │   ├── layouts/  
│   │   ├── pages/  
│   │   ├── tools/  
│   │   ├── real-cost/  
│   │   ├── battery-runtime/  
│   │   ├── qr-generator/  
│   │   ├── temporary-share/  
│   │   └── domain/  
└──
```

```

| | | repositories/
| | | adapters/
| | | | firebase/
| | | | supabase/
| | | storage/
| | | security/
| | | utils/
| | public/
| | tests/
| packages/
| | ui/
| | schemas/
| | config/
| docs/
| .github/workflows/
| package.json
| tsconfig.json
| README.md

```

Regla: una herramienta no debe importar directamente un SDK de Firebase/Supabase dentro de su función de cálculo. Debe hablar con una interfaz de repositorio/adaptador.

4. 📁 Interfaces fundamentales

```

export interface Tool {
  id: string;
  version: string;
  execute(input: I): O;
}

export interface TemporaryRepository {
  create(data: T, expiresAt: Date): Promise;
  get(id: string): Promise;
  delete(id: string): Promise;
}

export interface AuthProvider {
  signIn(): Promise;
  signOut(): Promise;
  currentUser(): Promise;
}

```

5. 📁 Herramientas iniciales

- 📁 Real Cost Calculator — cálculo local de coste total y margen.
- 📁 Battery Runtime Calculator — autonomía aproximada con advertencias sobre supuestos.
- 📁 QR Generator — generación local, sin guardar el contenido.
- 📁 Temporary Share — primer caso real de persistencia y expiración.

Las tres primeras deben funcionar sin cuenta y sin backend. Solo Temporary Share justificará almacenamiento remoto.

6. 📁 Ejemplo de módulo local

```

export type RealCostInput = {
  purchase: number;
  shipping: number;
  commission: number;
  taxes: number;
  packaging: number;
  advertising: number;
};

export function calculateRealCost(i: RealCostInput) {
  const total =
    i.purchase + i.shipping + i.commission +
    i.taxes + i.packaging + i.advertising;

  return { total };
}

```

Debe incluir validación: números finitos, valores no negativos y límites razonables de entrada.

7. 📄 Temporary Share: flujo técnico

```
CLIENTE
↓ POST /create
VALIDACIÓN
↓
GENERAR ID ALEATORIO
↓
GUARDAR PAYLOAD + expiresAt
↓
DEVOLVER URL
↓
USUARIO COMPARTE
↓
GET /share/:id
↓
¿expired? — sí → 404
|
no
↓
DEVOLVER CONTENIDO
↓
incrementar views / aplicar maxViews
↓
expirar / eliminar
```

Para contenido sensible, el MVP debe evitar prometer privacidad absoluta. Si se implementa cifrado extremo a extremo posteriormente, las claves no deben almacenarse junto al contenido de forma que anulen el propósito del cifrado.

8. 📄 Modelo de datos temporal

```
type TemporaryItem = {
  id: string;
  payload: string;
  createdAt: string;
  expiresAt: string;
  maxViews?: number;
  views: number;
  ownerId?: string;
};
```

Índices mínimos: id para lectura directa y expiresAt para limpieza. No crear índices innecesarios.

9. ♻️ Política de TTL

- Cada registro temporal debe tener expiresAt desde su creación.
- El backend debe rechazar elementos vencidos aunque el job de limpieza todavía no los haya eliminado físicamente.
- La limpieza debe ser idempotente: ejecutar dos veces no debe romper datos.
- Definir un TTL máximo, por ejemplo 7 días para el MVP.
- Definir límites de tamaño del payload.
- No almacenar archivos binarios en Temporary Share v1.0.

```
if (new Date() >= new Date(item.expiresAt)) {
  return null; // nunca entregar un elemento vencido
}
```

10. 📄 Seguridad y secretos

- Variables públicas de Firebase/Supabase pueden aparecer en el frontend cuando el proveedor las diseña así; las credenciales privilegiadas/service-role nunca.
- Reglas de seguridad deben existir antes de producción.
- Validar payloads con un esquema (por ejemplo Zod).

- Rate limiting para creación/lectura de enlaces temporales.
- No permitir HTML arbitrario sin sanitización.
- Aplicar Content Security Policy cuando sea viable.
- Registrar errores sin guardar contenido privado.

11. API conceptual

```
POST /api/temp
Body: { payload, ttlSeconds, maxViews? }
→ 201 { id, url, expiresAt }

GET /api/temp/:id
→ 200 { payload, expiresAt, remainingViews? }
→ 404 { error: "NOT_FOUND_OR_EXPIRED" }

DELETE /api/temp/:id
→ 204

GET /api/health
→ 200 { status: "ok" }
```

Los endpoints son conceptuales. Si el primer despliegue usa Firebase Functions, se adaptarán a callable/HTTP functions; si usa Supabase, pueden convertirse en Edge Functions. La interfaz pública debe permanecer estable.

12. Esquema SQL de referencia para Supabase

```
create table temporary_items (
  id text primary key,
  payload text not null,
  created_at timestamptz not null default now(),
  expires_at timestamptz not null,
  max_views integer,
  views integer not null default 0,
  owner_id uuid null
);

create index temporary_items_expires_at_idx
on temporary_items (expires_at);
```

En producción deben añadirse políticas RLS apropiadas y una estrategia de limpieza compatible con el proveedor.

13. Firebase: modelo equivalente

```
temporaryItems/{id}
payload: string
createdAt: timestamp
expiresAt: timestamp
maxViews: number | null
views: number
ownerId: string | null
```

La decisión final entre Firestore y Supabase debe hacerse después de validar el MVP. No se debe mantener doble escritura en ambos sistemas solo por “usar los tres”.

14. Estrategia de almacenamiento

- Nivel 0 — memoria del navegador: datos de formulario durante la sesión.
- Nivel 1 — LocalStorage: preferencias pequeñas y no sensibles.
- Nivel 2 — IndexedDB: datos locales mayores que deban persistir en el dispositivo.
- Nivel 3 — backend: solo datos que deban compartirse/sincronizarse.
- Nivel 4 — Storage de archivos: último recurso, nunca por defecto.

El proyecto debe medir tamaño de payload, cantidad de registros, egress y errores. La sostenibilidad se controla con métricas, no con suposiciones.

15. ☐ Requisitos mobile-first

- Interfaz usable desde 360 px de ancho.
- Botones táctiles grandes.
- No depender de hover.
- Inputs con teclado numérico cuando corresponda.
- Procesamiento local para evitar esperas innecesarias.
- Compatibilidad con navegadores Android modernos.
- Estados claros: calculando, listo, error, copiado, expirado.

16. ☐ Testing

```
describe("calculateRealCost", () => {  
  it("sums all cost components", () => {  
    expect(calculateRealCost({  
      purchase: 100,  
      shipping: 10,  
      commission: 5,  
      taxes: 5,  
      packaging: 2,  
      advertising: 3  
    }).total).toBe(125);  
  });  
});
```

Cada herramienta debe probar: caso normal, cero, valores extremos, entrada inválida y precisión numérica cuando corresponda.

17. ☐ CI/CD

```
push  
↓  
install  
↓  
lint  
↓  
typecheck  
↓  
unit tests  
↓  
build  
↓  
deploy
```

- Pull requests deben ejecutar lint + typecheck + tests + build.
- La rama principal solo recibe código que pase CI.
- Secrets solo en el entorno del proveedor/CI, nunca en Git.
- El build debe poder reproducirse desde un checkout limpio.

18. ☐ Observabilidad

- Tiempo de respuesta.
- Errores por endpoint.
- Cantidad de elementos temporales creados.
- Cantidad de elementos expirados.
- Tamaño promedio de payload.
- Uso de egress.
- Uso de almacenamiento.
- Tasa de errores del frontend.

No registrar el contenido de los mensajes temporales como métrica.

19. Preparación para monetización

- El núcleo gratuito debe ser útil sin cuenta.
- Premium puede desbloquear sincronización, historial, personalización y límites superiores.
- Publicidad solo en páginas públicas y donde no perjudique la herramienta.
- B2B puede utilizar paquetes especializados.
- No introducir pagos antes de validar que existe uso recurrente.

20. SEO técnico

- URLs estables por herramienta.
- Title y meta description únicos.
- Contenido explicativo real.
- Datos estructurados cuando correspondan.
- Sitemap.
- robots.txt correcto.
- Rendimiento y Core Web Vitals.
- Páginas de error útiles.

No generar cientos de páginas artificiales sin contenido sustancial. La herramienta debe ser el producto y el contenido debe explicar cómo y cuándo utilizarla.

21. Roadmap de implementación

Sprint 1 — Repositorio + Vite/TypeScript + UI base + routing + CI.

Sprint 2 — Real Cost + Battery Runtime + QR Generator.

Sprint 3 — Testing + SEO + accesibilidad + PWA opcional.

Sprint 4 — Temporary Share + backend elegido + TTL + rate limiting.

Sprint 5 — Autenticación opcional + dashboard mínimo.

Sprint 6 — Métricas + monetización experimental.

Sprint 7 — Nuevas herramientas basadas en uso real.

22. Definition of Done

- Código tipado y formateado.
- Tests pasando.
- Sin secretos en Git.
- Validación de entradas.
- Responsive en Android.
- Accesibilidad básica.
- Errores manejados.
- Documentación de la herramienta.
- Métricas mínimas.
- Coste de almacenamiento estimado.
- Política de retención definida.

- Build reproducible y desplegable.

23. ☐ Regla de oro para futuras IAs/desarrolladores

Antes de agregar Firebase, Supabase, Storage o cualquier API externa, responder obligatoriamente: “¿Por qué esta función no puede ejecutarse localmente?” Si existe una respuesta técnica válida, se añade el servicio. Si no existe, permanece local.

La segunda pregunta obligatoria es: “¿Qué dato estamos almacenando y durante cuánto tiempo?” Si no existe una respuesta concreta, ese dato no debe almacenarse.

24. ☐ Resultado técnico esperado

Al finalizar el MVP debe existir una aplicación web ligera donde el usuario pueda entrar sin cuenta, utilizar varias herramientas locales y, en un caso controlado, crear un recurso temporal compartible que expire automáticamente. El sistema debe demostrar que la arquitectura puede crecer sin convertir el almacenamiento en el centro del producto.

☐ **Próximo artefacto recomendado: Documento de Implementación v1.0: comandos exactos de creación del repositorio, archivos iniciales completos, configuración Vite/TypeScript, ESLint/Prettier, GitHub Actions y código del primer módulo. A partir de ahí ya se puede pasar de la arquitectura a construir Andryus Lab de verdad.** ☐